Exercise 1

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/sleep.h>
#include <util/delay.h>

#define LED_PIN PD5

volatile uint8_t timer_fired = 0;

ISR(TIMER1_COMPA_vect) {
    timer_fired = 1;
}

void setup_timer1(void) {
    DDRD |= (1 << LED_PIN);

    TCCR1B |= (1 << WGM12);

    OCR1A = 15624;

    TCCR1B |= (1 << CS12) | (1 << CS10);

    TIMSK1 |= (1 << OCIE1A);
}

void main(void) {
    setup_timer1();

    set_sleep_mode(SLEEP_MODE_IDLE);

    sei();

    while (1) {
        if (timer_fired) {
            PORTD |= (1 << LED_PIN);
            _delay_ms(100);
            PORTD &= ~(1 << LED_PIN);

            timer_fired = 0;
        }

        sleep_enable();
```

```
        sleep_cpu();
        sleep_disable();
    }
}
```

Exercise 2

```
#include <avr/io.h>
#include <util/delay.h>

#define LED_PIN PD5
#define ON_TIME_MS 100
#define PERIOD_MS 1000

void main(void) {
    DDRD |= (1 << LED_PIN);

    while (1) {
        PORTD |= (1 << LED_PIN);
        _delay_ms(ON_TIME_MS);

        PORTD &= ~(1 << LED_PIN);
        _delay_ms(PERIOD_MS - ON_TIME_MS);
    }
}
```

Comparison: E_{avg_sleep} will be significantly lower than E_{avg_normal}, demonstrating the power saving of using sleep modes.

Exercise 3

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/sleep.h>

#define LED_PIN PD5

volatile uint8_t timer1_fired = 0;

ISR(TIMER1_COMPA_vect) {
    timer1_fired = 1;
```

```
}

void setup_timer1(void) {
    DDRD |= (1 << LED_PIN);
    TCCR1B |= (1 << WGM12);
    OCR1A = 15624;
    TCCR1B |= (1 << CS12) | (1 << CS10);
    TIMSK1 |= (1 << OCIE1A);
}

ISR(TIMER0_COMPA_vect) {
    TCCR0B &= ~((1 << CS02) | (1 << CS01) | (1 << CS00));
    TIMSK0 &= ~(1 << OCIE0A);
    PORTD &= ~(1 << LED_PIN);
}

void start_timer0(void) {
    TCNT0 = 0;

    TCCR0A |= (1 << WGM01);

    OCR0A = 97;

    TCCR0B |= (1 << CS02) | (1 << CS00);

    TIMSK0 |= (1 << OCIE0A);
}

void main(void) {
    setup_timer1();
    set_sleep_mode(SLEEP_MODE_IDLE);
    sei();

    while (1) {
        if (timer1_fired) {
            PORTD |= (1 << LED_PIN);
            start_timer0();

            timer1_fired = 0;
        }

        sleep_enable();
        sleep_cpu();
        sleep_disable();
    }
```

```
}
```

Exercise 4

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include <avr/sleep.h>
#include <avr/power.h>
#define LED_PIN PD5

ISR(INT0_vect) {
}

void setup_external_int0(void) {
    PORTD |= (1 << PD2);
    EICRA |= (1 << ISC01);
    EICRA &= ~(1 << ISC00);
    EIMSK |= (1 << INT0);
}

void setup_adc(void) {
    power_adc_disable();
    ADMUX &= ~((1 << MUX3) | (1 << MUX2) | (1 << MUX1) | (1 << MUX0));
    ADCSRA |= (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);
}

uint16_t read_adc(uint8_t channel) {
    power_adc_enable();

    ADMUX = (ADMUX & 0xF0) | (channel & 0x0F);
    ADCSRA |= (1 << ADSC);
    while (ADCSRA & (1 << ADSC));
    uint16_t result = ADC;
    power_adc_disable();
    return result;
}


void main(void) {
    DDRD |= (1 << LED_PIN);
    setup_external_int0();
    setup_adc();
    set_sleep_mode(SLEEP_MODE_PWR_DOWN);
    sei();
```

```
    while (1) {
        PORTD &= ~(1 << LED_PIN);

        sleep_enable();
        sleep_cpu();
        sleep_disable();

        PORTD |= (1 << LED_PIN);
        uint16_t analog_value = read_adc(0); // Read A0

    }
}
```

Github Link:

https://github.com/Djaber-DB/cyber-physical-systems